Wrangling, Summarizing, and Plotting

Preliminaries

This second tutorial goes a little bit further into ways to use `pandas` for taking early looks at and plotting data. Think about the many questions you might ask as you explore data.

Think about the many types of explorative plots or summary statistics you might need to ask over and over again on eahc new dataset. You might want to group datapoints, or make early statistics like a *t-test* or make a plot of the distributions in the data.

`Pandas` provides ways to easily plot, summarize and organize the data so as to get you closer to answering initial questions on data.

In this tutorial, we will learn to analyze data by group. Specifically we will cover:

- the `split-apply-combine` concept
- grouping using `groupby()`
- doing simple grouped calculations and statistics
- doing multiple calculations with `agg()`
- `multiIndexing`
- simple summaries with `pivot` tables

We will use the dataset from before and load both `pandas` and `numpy`.

In [1]:
```
import numpy as np
import pandas as pd
```

Let's re-use our function to load and clean up data from last time (but with a new name).

In [2]:
```
def bcd_load_clean():
    bcd = pd.read_csv('./data/breast_cancer_data.csv')
    bcd['patient_id'] = bcd['patient_id'].astype('string')
    bcd['doctor_name'] = bcd['doctor_name'].str.split().str[1]
    bcd['bare_nuclei'] = bcd['bare_nuclei'].replace('?', '')
    bcd['bare_nuclei'] = pd.to_numeric(bcd['bare_nuclei'])

    return bcd
```

Load our data:

In [3]:
```
bcd = bcd_load_clean()
```

Now, for convenience, let's make a smaller data set to play with. We'll do this by dropping some of the columns.

We can do this in one of two ways. We can either `.drop` the columns we don't want, or `.copy` the columns we do. Here's the first method:

In [4]:
```
bcd2 = bcd.drop(labels = ['patient_id', 'cell_size_uniformity', 'cell_shape_
                          'cell_shape_uniformity', 'marginal_adhesion', 'sir
                          'bare_nuclei', 'normal_nucleoli', 'mitoses'],
                axis = 1,             # we're selecting column - default is
                inplace = False)      # we could modify bcd itself with True
```

In [5]:
```
bcd2
```

Out[5]:

	clump_thickness	bland_chromatin	class	doctor_name
0	5.0	3.0	benign	Doe
1	5.0	3.0	benign	Smith
2	3.0	3.0	benign	Lee
3	6.0	3.0	benign	Smith
4	4.0	3.0	benign	Wong
...	...	...	...	...
694	3.0	1.0	benign	Lee
695	2.0	1.0	benign	Smith
696	5.0	8.0	malignant	Lee
697	4.0	10.0	malignant	Lee
698	4.0	10.0	malignant	Wong

699 rows × 4 columns

In the cell below, make the same new data frame using column indexing and the `.copy()` method.

In [9]:
```
 # make new bcd2 using .copy()
bcd3 = bcd[['clump_thickness','bland_chromatin','class','doctor_name']].copy
```

In [10]:
```
 # look at new bcd2
bcd3
```

Out[10]:

	clump_thickness	bland_chromatin	class	doctor_name
0	5.0	3.0	benign	Doe
1	5.0	3.0	benign	Smith
2	3.0	3.0	benign	Lee
3	6.0	3.0	benign	Smith
4	4.0	3.0	benign	Wong
...	...	...	...	...
694	3.0	1.0	benign	Lee
695	2.0	1.0	benign	Smith
696	5.0	8.0	malignant	Lee
697	4.0	10.0	malignant	Lee
698	4.0	10.0	malignant	Wong

699 rows × 4 columns

What might we want from this data set?

The main thing that comes to mind is whether any of the measures are related to the kind of tumor. To do this, we can

- group the data by the "class" column
- perform some operation, like computing the mean, separately for the groups.

We might also want to see if the doctors are behaving consistently with respect to one another.

The split-apply-combine workflow

Much of data wrangling can be thought of "split-apply-combine". This is where we

- *split* the data into groups
- do ("*apply*") some function or manipulation on a per-group basis
- *combine* the results back into a data frame, series, etc.

Happily, the "combine" step is often handled for you by the methods that do the "apply" step.

Splitting - the `groupby()` method

Grouping the data is easy using the `groupby()` method. We just provide the name of a grouping variable. Since the main question at hand is how the measurements might relate to the type of tumor, Let's group by tumor "class".

In [11]:
```
grpd = bcd2.groupby('class')
```

If we try to look at it:

In [12]:
```
grpd
```

Out[12]:
```
<pandas.core.groupby.generic.DataFrameGroupBy object at 0x12e83bb50>
```

we see that the output of '.groupby() isn't a regular data frame, but rather a `DataFrameGroupBy` object. To interegate it, well need to use its methods or look at its attributes.

In the cell below, use the <TAB> key trick to browse the methods and properties that `grpd` has.

In [14]:
```
grpd.describe()
```

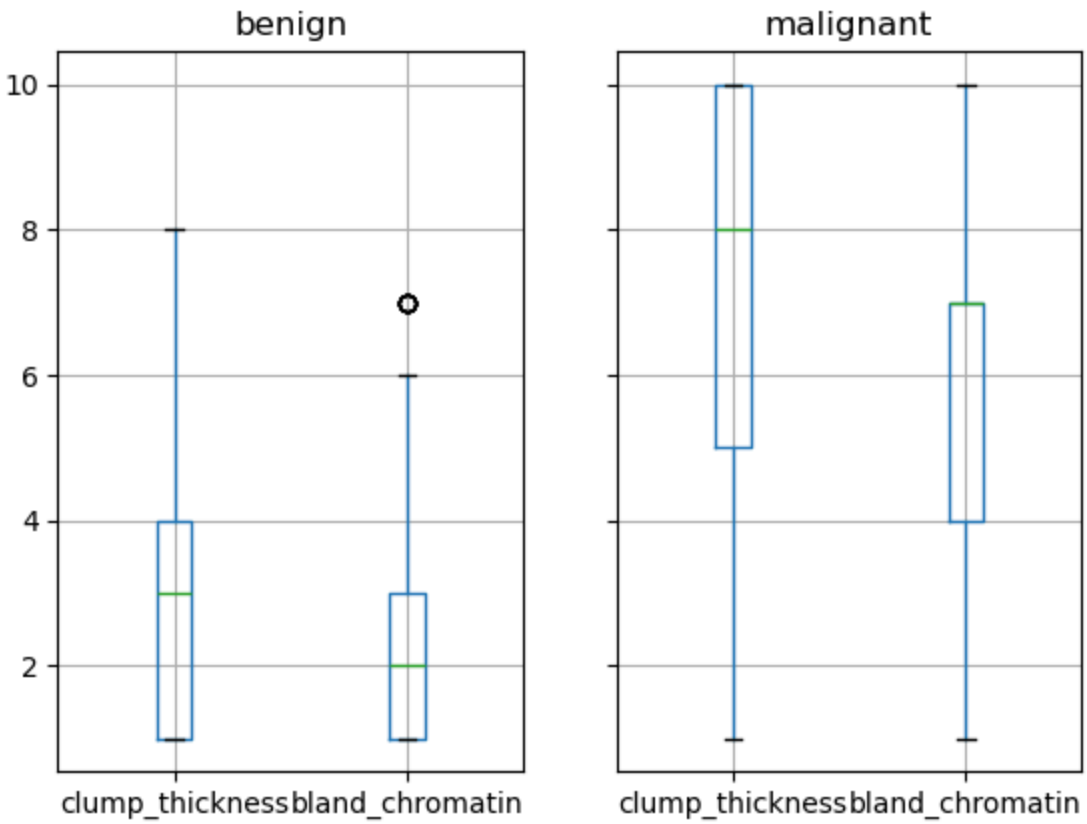
Out[14]:

									clump_thickness	
	count	mean	std	min	25%	50%	75%	max	count	mean
class										
benign	458.0	2.956332	1.674318	1.0	1.0	3.0	4.0	8.0	455.0	2.105495
malignant	240.0	7.204167	2.429763	1.0	5.0	8.0	10.0	10.0	240.0	5.991667

One of the early methods on the list is `.boxplot` - see what that does in the cell below!

In [15]:
```
grpd.boxplot()
```

Out[15]:
```
benign            Axes(0.1,0.15;0.363636x0.75)
malignant    Axes(0.536364,0.15;0.363636x0.75)
dtype: object
```



While not the prettiest plot in the world, it does give us a hint that both of these variables might be related to tumor type.

(Make sure you remember or remind yourself what a box shows you.)

Now let's see if we can `.describe` the grouped data using the cell below.

In [16]:
```
# some useful summary numbers
grpd.describe()
```

Out[16]:

	clump_thickness									
	count	mean	std	min	25%	50%	75%	max	count	mean
class										
benign	458.0	2.956332	1.674318	1.0	1.0	3.0	4.0	8.0	455.0	2.105495
malignant	240.0	7.204167	2.429763	1.0	5.0	8.0	10.0	10.0	240.0	5.991667

What is the approximate mean difference between the groups for each of the two measures? (no coding - just eyeball it)

~4.2

Do a very rough guesstimate of Student's *t* for the clump thickness (e.g., just use the larger `std` and smaller `countn`). You can use the cell below as a calculator if you like.

In [19]:
```
mean_benign = 2.956332
mean_malignant = 7.204167
std_larger = 2.429763
n_smaller = 240

difference = mean_malignant - mean_benign
se = np.sqrt(2 * (std_larger**2) / n_smaller)
t = difference / se
t
```

Out[19]:
```
19.151127497290542
```

What does that tell you?

There's a stark difference in the clump thickness in benign and malignant types

---

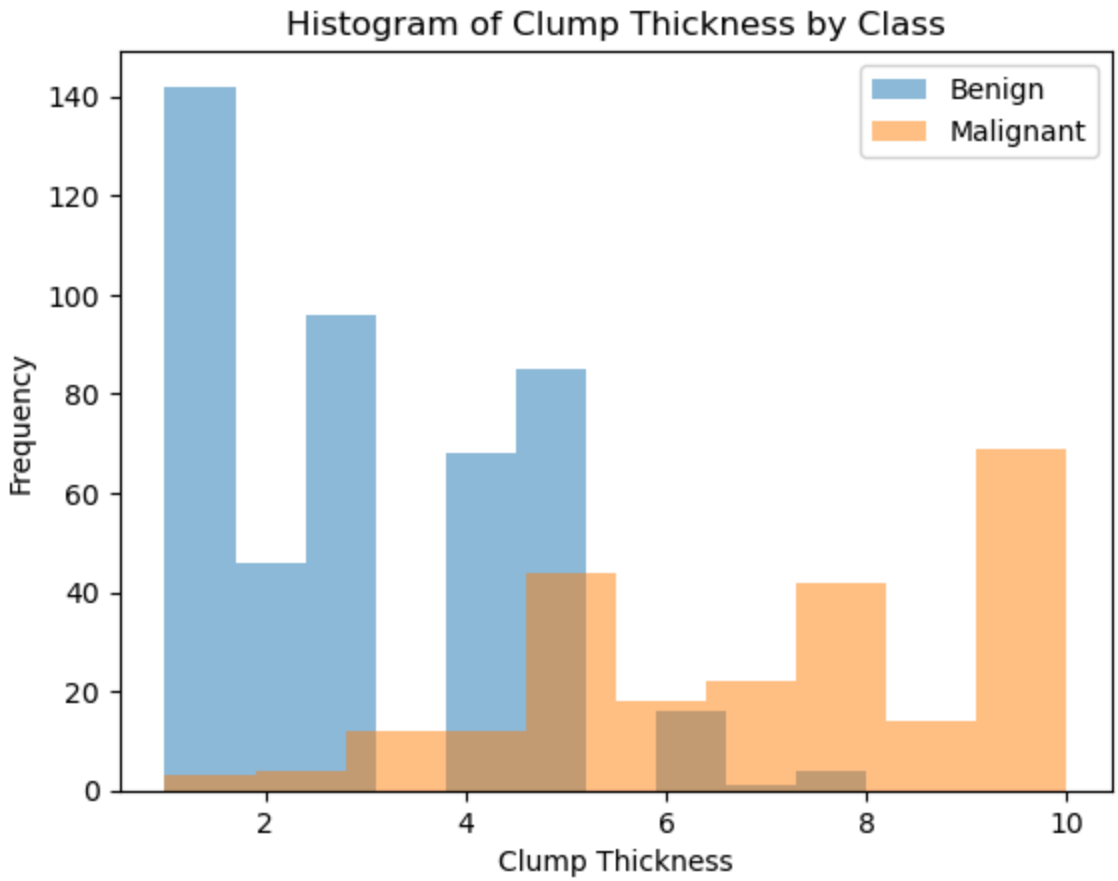
Now let's make a histogram of grouped data using the appropriate method.

In [23]:
```
import matplotlib.pyplot as plt

benign = grpd.get_group("benign")["clump_thickness"]
malignant = grpd.get_group("malignant")["clump_thickness"]

plt.hist(benign, bins=10, alpha=0.5, label="Benign")
plt.hist(malignant, bins=10, alpha=0.5, label="Malignant")

plt.xlabel("Clump Thickness")
plt.ylabel("Frequency")
plt.title("Histogram of Clump Thickness by Class")
plt.legend()
plt.show()
```



Again, not the prettiest plot in the world. By comparison with the boxplot above, we can see that the top row of these histograms correspond to the benign tumors. We can also see that, consistent with the box plots, there is quite a bit overlap in the data values across groups. Is this consistent with your calculation of *t*?

Generally it looks like it is

We can also get the data for a specific group out of the grouped object. This will return a regular data frame the same width as the original, but only containing the requested group's data.

In [24]:
```
grpd.get_group('benign')
```

Out[24]:

	clump_thickness	bland_chromatin	class	doctor_name
0	5.0	3.0	benign	Doe
1	5.0	3.0	benign	Smith
2	3.0	3.0	benign	Lee
3	6.0	3.0	benign	Smith
4	4.0	3.0	benign	Wong
...	...	...	...	...
690	1.0	1.0	benign	Doe
692	3.0	1.0	benign	Wong
693	3.0	2.0	benign	Lee
694	3.0	1.0	benign	Lee
695	2.0	1.0	benign	Smith

458 rows × 4 columns

In the cell below, confirm that the returned object is indeed a pandas `DataFrame`.

In [25]:
```
type(grpd.get_group("benign"))
```

Out[25]:
```
pandas.core.frame.DataFrame
```

Applying - doing things to the data within groups

Once we have grouped data, we can easily caluculate things per group. Using the <TAB> trick, we can see that these objects produced by `groupby()` have methods for all the common statistical summaries.

Simple calculations

We can compute the mean for each measure *by group*

In [26]:
```
my_means = grpd.mean(numeric_only = True)
my_means
```

Out[26]:

class	clump_thickness	bland_chromatin
benign	2.956332	2.105495
malignant	7.204167	5.991667

In the cells below, compute and show the

standard deviations:

In [28]:
```
my_sds =  grpd.std(numeric_only = True)
my_sds
```

Out[28]:

class	clump_thickness	bland_chromatin
benign	1.674318	1.081417
malignant	2.429763	2.270406

and the counts

In [30]:
```
my_counts = grpd.count()
my_counts
```

Out[30]:

class	clump_thickness	bland_chromatin	doctor_name
benign	458	455	458
malignant	240	240	241

We can easily do simple maths on data frames of a compatible size. Here's a comparison of how many z-scores above zero each of the means are:

In [31]:
```
my_zeds = my_means/my_sds
my_zeds
```

Out[31]:

class	clump_thickness	bland_chromatin
benign	1.765693	1.946977
malignant	2.964967	2.639029

We can look at the difference between the z-scores for each measure using the `diff()` method, which takes the first difference down the rows.

In [32]:
```
my_zeds.diff()
```

Out[32]:

class	clump_thickness	bland_chromatin
benign	NaN	NaN
malignant	1.199274	0.692052

This difference in z-scores – how far apart two means are in terms of the standard deviation of the data – is roughly what statisticians call "effect size".

Why do the NaNs appear in the first row?

because it computes teh difference with the previous row

In the cell below, extract just the row with the numbers from `my_zeds` into a new series. (hint: using `.loc` is probably easiest)

In [33]:
```
z_row = my_zeds.loc["malignant"]

z_row
```

Out[33]:
```
clump_thickness    2.964967
bland_chromatin    2.639029
Name: malignant, dtype: float64
```

We could also compute the difference by using `.loc[]` row indexing and simple maths.

In [34]:
```
my_zeds.loc['malignant'] - my_zeds.loc['benign']
```

Out[34]:
```
clump_thickness    1.199274
bland_chromatin    0.692052
dtype: float64
```

Mulitiple caclulations with `agg()`

We can do multiple calculation at once by placing function names inside the `agg()`) or `aggregate()`) methods (they are synonyms). Here's where importing numpy comes in handy.

In [35]:
```
grpd.agg([np.mean, np.std])
```

```
/var/folders/71/br_33dt53n948nsmj9czsvr80000gn/T/ipykernel_59225/1757925766.
py:1: FutureWarning: The provided callable <function mean at 0x10951d440> is
currently using SeriesGroupBy.mean. In a future version of pandas, the provi
ded callable will be used directly. To keep current behavior pass the string
"mean" instead.
  grpd.agg([np.mean, np.std])
/var/folders/71/br_33dt53n948nsmj9czsvr80000gn/T/ipykernel_59225/1757925766.
py:1: FutureWarning: The provided callable <function std at 0x10951d580> is
currently using SeriesGroupBy.std. In a future version of pandas, the provid
ed callable will be used directly. To keep current behavior pass the string
"std" instead.
  grpd.agg([np.mean, np.std])
```

```
---------------------------------------------------------------------------
TypeError                                 Traceback (most recent call last)
File /opt/anaconda3/lib/python3.11/site-packages/pandas/core/groupby/groupb
y.py:1874, in GroupBy._agg_py_fallback(self, how, values, ndim, alt)
   1873 try:
-> 1874     res_values = self.grouper.agg_series(ser, alt, preserve_dtype=Tr
ue)
   1875 except Exception as err:

File /opt/anaconda3/lib/python3.11/site-packages/pandas/core/groupby/ops.py:
849, in BaseGrouper.agg_series(self, obj, func, preserve_dtype)
    847     preserve_dtype = True
--> 849 result = self._aggregate_series_pure_python(obj, func)
    851 if len(obj) == 0 and len(result) == 0 and isinstance(obj.dtype, Exte
nsionDtype):

File /opt/anaconda3/lib/python3.11/site-packages/pandas/core/groupby/ops.py:
877, in BaseGrouper._aggregate_series_pure_python(self, obj, func)
    876 for i, group in enumerate(splitter):
--> 877     res = func(group)
    878     res = extract_result(res)

File /opt/anaconda3/lib/python3.11/site-packages/pandas/core/groupby/groupb
y.py:2380, in GroupBy.mean.<locals>.<lambda>(x)
   2377 else:
   2378     result = self._cython_agg_general(
   2379         "mean",
-> 2380         alt=lambda x: Series(x).mean(numeric_only=numeric_only),
   2381         numeric_only=numeric_only,
   2382     )
   2383     return result.__finalize__(self.obj, method="groupby")

File /opt/anaconda3/lib/python3.11/site-packages/pandas/core/series.py:6225,
in Series.mean(self, axis, skipna, numeric_only, **kwargs)
   6217 @doc(make_doc("mean", ndim=1))
   6218 def mean(
   6219     self,
   (...)
   6223     **kwargs,
   6224 ):
-> 6225     return NDFrame.mean(self, axis, skipna, numeric_only, **kwargs)

File /opt/anaconda3/lib/python3.11/site-packages/pandas/core/generic.py:1199
2, in NDFrame.mean(self, axis, skipna, numeric_only, **kwargs)
  11985 def mean(
  11986     self,
  11987     axis: Axis | None = 0,
   (...)
  11990     **kwargs,
  11991 ) -> Series | float:
> 11992     return self._stat_function(
  11993         "mean", nanops.nanmean, axis, skipna, numeric_only, **kwargs
  11994     )

File /opt/anaconda3/lib/python3.11/site-packages/pandas/core/generic.py:1194
9, in NDFrame._stat_function(self, name, func, axis, skipna, numeric_only, *
```

```
*kwargs)
  11947 validate_bool_kwarg(skipna, "skipna", none_allowed=False)
> 11949 return self._reduce(
  11950     func, name=name, axis=axis, skipna=skipna, numeric_only=numeric_
only
  11951 )

File /opt/anaconda3/lib/python3.11/site-packages/pandas/core/series.py:6133,
in Series._reduce(self, op, name, axis, skipna, numeric_only, filter_type, *
*kwds)
   6129     raise TypeError(
   6130         f"Series.{name} does not allow {kwd_name}={numeric_only} "
   6131         "with non-numeric dtypes."
   6132     )
-> 6133 return op(delegate, skipna=skipna, **kwds)

File /opt/anaconda3/lib/python3.11/site-packages/pandas/core/nanops.py:147,
in bottleneck_switch.__call__.<locals>.f(values, axis, skipna, **kwds)
    146 else:
--> 147     result = alt(values, axis=axis, skipna=skipna, **kwds)
    149 return result

File /opt/anaconda3/lib/python3.11/site-packages/pandas/core/nanops.py:404,
in _datetimelike_compat.<locals>.new_func(values, axis, skipna, mask, **kwar
gs)
    402     mask = isna(values)
--> 404 result = func(values, axis=axis, skipna=skipna, mask=mask, **kwargs)
    406 if datetimelike:

File /opt/anaconda3/lib/python3.11/site-packages/pandas/core/nanops.py:720,
in nanmean(values, axis, skipna, mask)
    719 the_sum = values.sum(axis, dtype=dtype_sum)
--> 720 the_sum = _ensure_numeric(the_sum)
    722 if axis is not None and getattr(the_sum, "ndim", False):

File /opt/anaconda3/lib/python3.11/site-packages/pandas/core/nanops.py:1693,
in _ensure_numeric(x)
   1691 if isinstance(x, str):
   1692     # GH#44008, GH#36703 avoid casting e.g. strings to numeric
-> 1693     raise TypeError(f"Could not convert string '{x}' to numeric")
   1694 try:

TypeError: Could not convert string 'DoeSmithLeeSmithWongDoeSmithSmithDoeDoe
SmithWongLeeSmithWongSmithSmithDoeLeeDoeDoeSmithLeeSmithLeeLeeWongWongSmithS
mithLeeWongWongWongDoeSmithWongWongDoeWongDoeLeeWongWongWongLeeWongWongDoeLe
eLeeLeeSmithLeeDoeDoeLeeWongDoeLeeDoeWongDoeLeeDoeLeeWongDoeSmithLeeWongLeeD
oeWongLeeLeeWongLeeDoeSmithSmithWongWongWongLeeLeeLeeSmithLeeDoeDoeLeeLeeDoe
SmithLeeLeeLeeLeeDoeDoeDoeLeeLeeWongSmithLeeDoeWongSmithWongLeeDoeWongSmithL
eeWongDoeSmithWongDoeLeeLeeDoeWongWongLeeDoeLeeDoeDoeDoeSmithWongSmithDoeSmi
thSmithWongWongDoeWongWongDoeLeeSmithLeeDoeWongSmithLeeDoeWongDoeWongSmithDo
eSmithDoeLeeLeeSmithSmithDoeWongWongLeeSmithLeeWongSmithDoeWongSmithWongSmit
hDoeLeeDoeSmithWongSmithWongWongWongDoeLeeDoeSmithDoeDoeWongDoeDoeSmithWongL
eeDoeDoeLeeWongDoeSmithSmithWongLeeSmithDoeDoeDoeSmithLeeDoeLeeDoeSmithWongL
eeLeeSmithWongWongDoeLeeSmithDoeSmithDoeLeeWongDoeDoeWongWongDoeWongWongLeeW
ongLeeWongSmithLeeSmithLeeLeeWongLeeSmithWongSmithDoeWongWongSmithLeeWongLee
SmithDoeLeeLeeDoeDoeDoeDoeSmithLeeWongDoeLeeLeeSmithLeeLeeSmithLeeWongSmithS
```

```
mithDoeWongDoeWongSmithDoeLeeDoeLeeLeeLeeDoeWongDoeWongDoeLeeDoeDoeDoeWongLe
eDoeLeeDoeSmithSmithLeeSmithWongSmithDoeLeeSmithWongDoeDoeDoeLeeDoeLeeSmithD
oeSmithDoeWongSmithWongDoeSmithSmithWongWongLeeLeeDoeDoeLeeSmithWongLeeLeeLe
eLeeLeeLeeLeeSmithSmithLeeDoeLeeSmithDoeDoeLeeDoeLeeSmithSmithWongWongSmithS
mithSmithLeeSmithWongDoeWongDoeDoeSmithDoeDoeWongWongWongWongWongDoeLeeSmith
SmithSmithLeeDoeSmithDoeLeeSmithDoeWongSmithSmithDoeLeeWongDoeSmithWongWongW
ongLeeDoeSmithSmithLeeLeeDoeDoeSmithWongWongSmithDoeSmithLeeDoeDoeDoeSmithDo
eSmithDoeWongSmithSmithLeeSmithDoeDoeWongLeeLeeWongLeeWongLeeWongDoeWongLeeL
eeDoeWongLeeWongDoeDoeLeeSmithLeeDoeWongLeeLeeSmith' to numeric

The above exception was the direct cause of the following exception:

TypeError                                 Traceback (most recent call last)
Cell In[35], line 1
----> 1 grpd.agg([np.mean, np.std])

File /opt/anaconda3/lib/python3.11/site-packages/pandas/core/groupby/generi
c.py:1445, in DataFrameGroupBy.aggregate(self, func, engine, engine_kwargs,
*args, **kwargs)
   1442     kwargs["engine_kwargs"] = engine_kwargs
   1444 op = GroupByApply(self, func, args=args, kwargs=kwargs)
-> 1445 result = op.agg()
   1446 if not is_dict_like(func) and result is not None:
   1447     # GH #52849
   1448     if not self.as_index and is_list_like(func):

File /opt/anaconda3/lib/python3.11/site-packages/pandas/core/apply.py:178, i
n Apply.agg(self)
    175     return self.agg_dict_like()
    176 elif is_list_like(func):
    177     # we require a list, but not a 'str'
--> 178     return self.agg_list_like()
    180 if callable(func):
    181     f = com.get_cython_func(func)

File /opt/anaconda3/lib/python3.11/site-packages/pandas/core/apply.py:311, i
n Apply.agg_list_like(self)
    303 def agg_list_like(self) -> DataFrame | Series:
    304     """
    305     Compute aggregation in the case of a list-like argument.
    306
   (...)
    309     Result of aggregation.
    310     """
--> 311     return self.agg_or_apply_list_like(op_name="agg")

File /opt/anaconda3/lib/python3.11/site-packages/pandas/core/apply.py:1353,
in GroupByApply.agg_or_apply_list_like(self, op_name)
   1348 # Only set as_index=True on groupby objects, not Window or Resample
   1349 # that inherit from this class.
   1350 with com.temp_setattr(
   1351     obj, "as_index", True, condition=hasattr(obj, "as_index")
   1352 ):
-> 1353     keys, results = self.compute_list_like(op_name, selected_obj, kw
args)
   1354 result = self.wrap_results_list_like(keys, results)
```

```
   1355 return result

File /opt/anaconda3/lib/python3.11/site-packages/pandas/core/apply.py:370, i
n Apply.compute_list_like(self, op_name, selected_obj, kwargs)
    364 colg = obj._gotitem(col, ndim=1, subset=selected_obj.iloc[:, index])
    365 args = (
    366     [self.axis, *self.args]
    367     if include_axis(op_name, colg)
    368     else self.args
    369 )
--> 370 new_res = getattr(colg, op_name)(func, *args, **kwargs)
    371 results.append(new_res)
    372 indices.append(index)

File /opt/anaconda3/lib/python3.11/site-packages/pandas/core/groupby/generi
c.py:255, in SeriesGroupBy.aggregate(self, func, engine, engine_kwargs, *arg
s, **kwargs)
    253 kwargs["engine"] = engine
    254 kwargs["engine_kwargs"] = engine_kwargs
--> 255 ret = self._aggregate_multiple_funcs(func, *args, **kwargs)
    256 if relabeling:
    257     # columns is not narrowed by mypy from relabeling flag
    258     assert columns is not None  # for mypy

File /opt/anaconda3/lib/python3.11/site-packages/pandas/core/groupby/generi
c.py:360, in SeriesGroupBy._aggregate_multiple_funcs(self, arg, *args, **kwa
rgs)
    358     for idx, (name, func) in enumerate(arg):
    359         key = base.OutputKey(label=name, position=idx)
--> 360         results[key] = self.aggregate(func, *args, **kwargs)
    362 if any(isinstance(x, DataFrame) for x in results.values()):
    363     from pandas import concat

File /opt/anaconda3/lib/python3.11/site-packages/pandas/core/groupby/generi
c.py:268, in SeriesGroupBy.aggregate(self, func, engine, engine_kwargs, *arg
s, **kwargs)
    266 if cyfunc and not args and not kwargs:
    267     warn_alias_replacement(self, func, cyfunc)
--> 268     return getattr(self, cyfunc)()
    270 if maybe_use_numba(engine):
    271     return self._aggregate_with_numba(
    272         func, *args, engine_kwargs=engine_kwargs, **kwargs
    273     )

File /opt/anaconda3/lib/python3.11/site-packages/pandas/core/groupby/groupb
y.py:2378, in GroupBy.mean(self, numeric_only, engine, engine_kwargs)
   2371     return self._numba_agg_general(
   2372         grouped_mean,
   2373         executor.float_dtype_mapping,
   2374         engine_kwargs,
   2375         min_periods=0,
   2376     )
   2377 else:
-> 2378     result = self._cython_agg_general(
   2379         "mean",
   2380         alt=lambda x: Series(x).mean(numeric_only=numeric_only),
```

```
   2381         numeric_only=numeric_only,
   2382     )
   2383     return result.__finalize__(self.obj, method="groupby")

File /opt/anaconda3/lib/python3.11/site-packages/pandas/core/groupby/groupb
y.py:1929, in GroupBy._cython_agg_general(self, how, alt, numeric_only, min_
count, **kwargs)
   1926     result = self._agg_py_fallback(how, values, ndim=data.ndim, alt=
alt)
   1927     return result
-> 1929 new_mgr = data.grouped_reduce(array_func)
   1930 res = self._wrap_agged_manager(new_mgr)
   1931 out = self._wrap_aggregated_output(res)

File /opt/anaconda3/lib/python3.11/site-packages/pandas/core/internals/base.
py:336, in SingleDataManager.grouped_reduce(self, func)
    334 def grouped_reduce(self, func):
    335     arr = self.array
--> 336     res = func(arr)
    337     index = default_index(len(res))
    339     mgr = type(self).from_array(res, index)

File /opt/anaconda3/lib/python3.11/site-packages/pandas/core/groupby/groupb
y.py:1926, in GroupBy._cython_agg_general.<locals>.array_func(values)
   1923 else:
   1924     return result
-> 1926 result = self._agg_py_fallback(how, values, ndim=data.ndim, alt=alt)
   1927 return result

File /opt/anaconda3/lib/python3.11/site-packages/pandas/core/groupby/groupb
y.py:1878, in GroupBy._agg_py_fallback(self, how, values, ndim, alt)
   1876     msg = f"agg function failed [how->{how},dtype->{ser.dtype}]"
   1877     # preserve the kind of exception that raised
-> 1878     raise type(err)(msg) from err
   1880 if ser.dtype == object:
   1881     res_values = res_values.astype(object, copy=False)

TypeError: agg function failed [how->mean,dtype->object]
```

That either produced an error, or worked but pandas still complained to us because `grpd` has the doctors' names in it, and we obviously can't compute the means and standard deviations of those!

In the cell below, repeat the above calculation without triggering the warning.

In [37]:
```
grpd.agg(lambda x: [np.mean(x), np.std(x)] if np.issubdtype(x.dtype, np.numb
```

Out[37]:

class	clump_thickness	bland_chromatin	doctor_name
benign	[2.9563318777292578, 1.6724890829694323]	[2.1054945054945056, 1.080228164478429]	None
malignant	[7.204166666666667, 2.4246957140135232]	[5.991666666666666, 2.265670737086237]	None

---

We can use the pandas versions of functions by placing them in quotes. This is handy because, for example, pandas has a `count` and numpy doesn't.

In [38]:
```
grpd[['clump_thickness', 'bland_chromatin']].agg([np.mean, 'std', 'count'])
```

```
/var/folders/71/br_33dt53n948nsmj9czsvr80000gn/T/ipykernel_59225/3528688385.
py:1: FutureWarning: The provided callable <function mean at 0x10951d440> is
currently using SeriesGroupBy.mean. In a future version of pandas, the provi
ded callable will be used directly. To keep current behavior pass the string
"mean" instead.
  grpd[['clump_thickness', 'bland_chromatin']].agg([np.mean, 'std', 'coun
t'])
```

Out[38]:

	clump_thickness			bland_chromatin		
	mean	std	count	mean	std	count
class						
benign	2.956332	1.674318	458	2.105495	1.081417	455
malignant	7.204167	2.429763	240	5.991667	2.270406	240

MultiIndexing - getting at our summary data

Let's store our summary table little summary table above in its own data frame. This is going to complete our *split-apply-combine* by creating and naming a `DataFrame` object.

In [39]:
```
my_summary = grpd[['clump_thickness', 'bland_chromatin']].agg([np.mean, 'stc
my_summary
```

```
/var/folders/71/br_33dt53n948nsmj9czsvr80000gn/T/ipykernel_59225/2057379153.
py:1: FutureWarning: The provided callable <function mean at 0x10951d440> is
currently using SeriesGroupBy.mean. In a future version of pandas, the provi
ded callable will be used directly. To keep current behavior pass the string
"mean" instead.
  my_summary = grpd[['clump_thickness', 'bland_chromatin']].agg([np.mean, 's
td', 'count'])
```

Out[39]:

	clump_thickness			bland_chromatin		
	mean	std	count	mean	std	count
class						
benign	2.956332	1.674318	458	2.105495	1.081417	455
malignant	7.204167	2.429763	240	5.991667	2.270406	240

Notice that this data frame has hierachical column labels. In other words, there is a "clump_thickness" meta-column that contains three colums of its own, and that these subcolumns have the same names as those in the other meta-column. Thus saying "look at the mean column" would be ambiguous because the meta-column wasn't specified.

In pandas, this is known at "multiIndexing".

Getting a meta-column is easy - it's just like getting a regular column from a data frame.

In [40]:
```
my_summary['clump_thickness']
```

Out[40]:

	mean	std	count
class			
benign	2.956332	1.674318	458
malignant	7.204167	2.429763	240

To get a subcolumn, we can index the meta-column, and then index the subcolumn from that.

In [41]:
```
my_summary['clump_thickness']['mean']
```

Out[41]:
```
class
benign       2.956332
malignant    7.204167
Name: mean, dtype: float64
```

If this looks confusing, consider the same thing broken up into two steps:

In [42]:
```
meta_c = my_summary['clump_thickness']
meta_c
```

Out[42]:

	mean	std	count
class			
benign	2.956332	1.674318	458
malignant	7.204167	2.429763	240

In [43]:
```
meta_c['mean']
```

Out[43]:
```
class
benign       2.956332
malignant    7.204167
Name: mean, dtype: float64
```

In the cell below, extract the mean and std of "bland_chromatin" in one go.

In [44]:
```
grpd["bland_chromatin"].agg([np.mean, np.std])
```

```
/var/folders/71/br_33dt53n948nsmj9czsvr80000gn/T/ipykernel_59225/1648149949.
py:1: FutureWarning: The provided callable <function mean at 0x10951d440> is
currently using SeriesGroupBy.mean. In a future version of pandas, the provi
ded callable will be used directly. To keep current behavior pass the string
"mean" instead.
  grpd["bland_chromatin"].agg([np.mean, np.std])
/var/folders/71/br_33dt53n948nsmj9czsvr80000gn/T/ipykernel_59225/1648149949.
py:1: FutureWarning: The provided callable <function std at 0x10951d580> is
currently using SeriesGroupBy.std. In a future version of pandas, the provid
ed callable will be used directly. To keep current behavior pass the string
"std" instead.
  grpd["bland_chromatin"].agg([np.mean, np.std])
```

Out[44]:

class	mean	std
benign	2.105495	1.081417
malignant	5.991667	2.270406

If we want values from a row, we need to get a bit more fancy and use `.loc`.

We can get a whole row using `df.loc[row_index(s)]`

In [45]:
```
my_summary.loc['benign']
```

Out[45]:
```
clump_thickness  mean       2.956332
                 std        1.674318
                 count    458.000000
bland_chromatin  mean       2.105495
                 std        1.081417
                 count    455.000000
Name: benign, dtype: float64
```

(note: that this gave us a hierarchical index!)

We can get a row and and a particular column with `df.loc[row_index(s), (metacolumn_index, subcolumn_index)]`

In [46]:
```
my_summary.loc['benign', ('clump_thickness', 'mean')]
```

Out[46]:
```
2.9563318777292578
```

We can also get bigger slices of the data with the colon `:` operator:

In [47]:
```
my_summary.loc['benign', ('clump_thickness', 'mean'):('bland_chromatin', 'st
```

Out[47]:
```
clump_thickness  mean       2.956332
                 std        1.674318
                 count    458.000000
bland_chromatin  mean       2.105495
                 std        1.081417
Name: benign, dtype: float64
```

In the cell below, extract the mean and std of the bland chromatin meta-column.

In [48]:
```
my_summary.loc["benign", "bland_chromatin"]
```

Out[48]:
```
mean       2.105495
std        1.081417
count    455.000000
Name: benign, dtype: float64
```

Simple caculations with pivot tables

Pivot tables are summary data with the levels of one variable running down the row names (the index), the levels of another running across the column names, and values populating the interior. This should be made concrete by making one with the `pivot_table()` method:

In [49]:
```
bcd2.pivot_table(index = 'doctor_name', columns = 'class', values = 'bland_c
```

Out[49]:

class	benign	malignant
doctor_name		
Doe	2.000000	5.456140
Lee	2.067227	6.150000
Smith	1.980392	6.459459
Wong	2.388889	5.714286

By default, `pivot_table()` computes the group (row x column) means, but we can compute any of the standard summary statistics we wish. We just specify it using the `aggfunc` argument:

In [ ]:
```
bcd2.pivot_table(index = 'doctor_name',
                 columns = 'class',

                 values = 'bland_chromatin',
                 aggfunc = 'std')
```

Out[ ]:

class	benign	malignant
doctor_name		
Doe	1.003992	2.260453
Lee	1.014564	2.121920
Smith	0.943769	2.330202
Wong	1.303004	2.263846

These are called "pivot tables" because their implementation makes it easy to pivot our view of the data summary.

In the cell below, "pivot" our view of the means so we have "class" down the rows, doctor name across the columns, and the means of clump thickness inside the table.

In [51]:
```
clump_means_pivot = bcd2.pivot_table(
    index='class',
    columns='doctor_name',
    values='clump_thickness',
    aggfunc='mean'
)

clump_means_pivot
```

Out[51]:

doctor_name	Doe	Lee	Smith	Wong
class				
benign	2.637795	2.983471	3.098039	3.166667
malignant	7.586207	6.600000	7.356164	7.265306

Because clumb thickness and bland chromatin only have the values 1 to 10, we could use either one as a grouping variable in a pivot table:

In [52]:
```
bcd2.pivot_table(index = 'clump_thickness', columns = 'class', values = 'bla
```

Out[52]:

class	benign	malignant
clump_thickness		
1.0	1.978723	5.666667
2.0	2.090909	5.000000
3.0	2.145833	5.500000
4.0	2.117647	7.916667
5.0	2.011765	5.818182
6.0	3.062500	6.666667
7.0	2.000000	5.818182
8.0	3.750000	5.761905
9.0	NaN	5.142857
10.0	NaN	6.147059

Notice the NaNs – no benign tumors have a thickness of 9 or 10.

Grouped plotting

As we have already seen, the seaborn package can take care of grouping for us – we just need to assign a grouping variable to color ('hue'), style, etc. And we already know how to do all this!

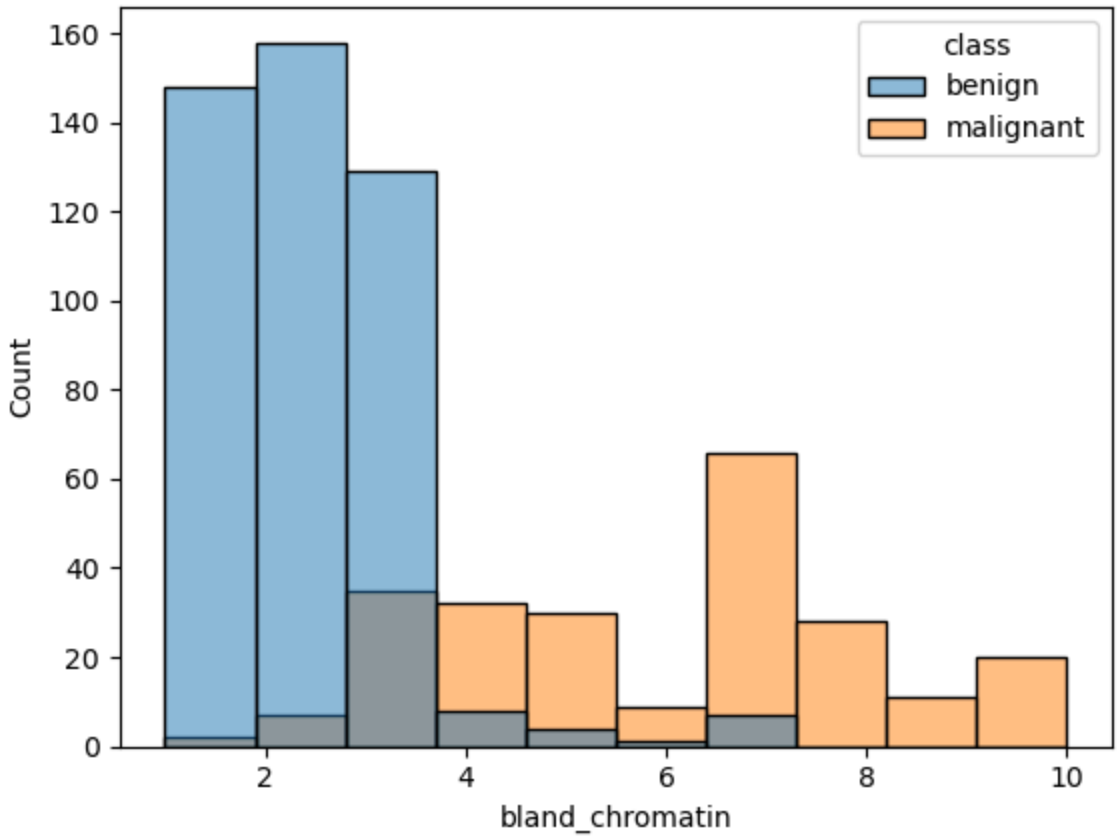
In [53]:
```
import seaborn as sns
```

Make a distribution plot (like a histogram) of bland chromatin values grouped by tumor type.

In [54]:
```
sns.histplot(data=bcd2, x="bland_chromatin", hue="class", kde=False, bins=10
```

```
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: Future
Warning: use_inf_as_na option is deprecated and will be removed in a future
version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```

Out[54]:
```
<Axes: xlabel='bland_chromatin', ylabel='Count'>
```

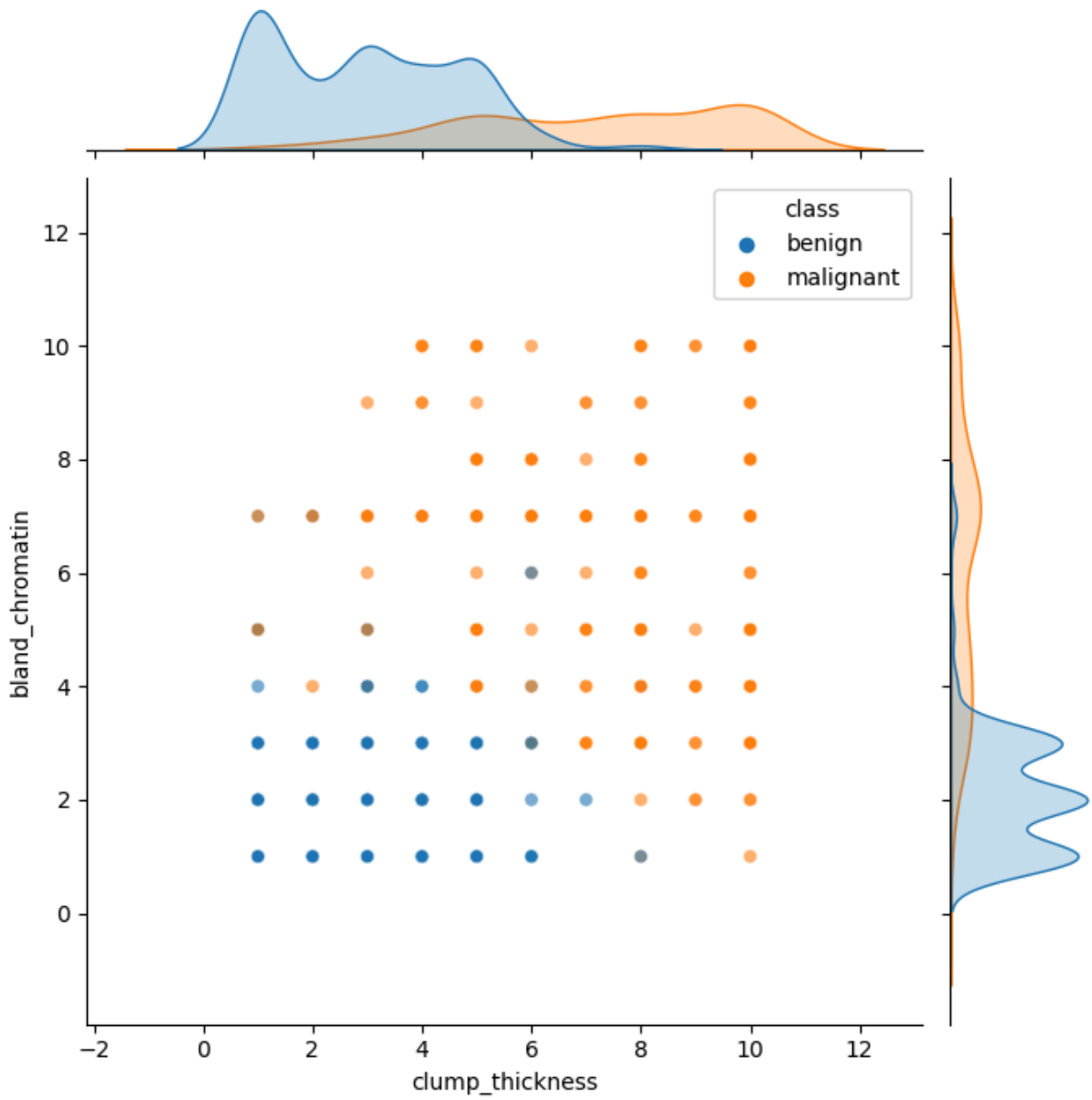


Make a joint plot grouped by tumor type:

In [55]:
```
sns.jointplot(data=bcd2, x="clump_thickness", y="bland_chromatin", hue="clas
    height=7, alpha=0.6)
```

```
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: Future
Warning: use_inf_as_na option is deprecated and will be removed in a future
version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: Future
Warning: use_inf_as_na option is deprecated and will be removed in a future
version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```

Out[55]:
```
<seaborn.axisgrid.JointGrid at 0x1759ed290>
```

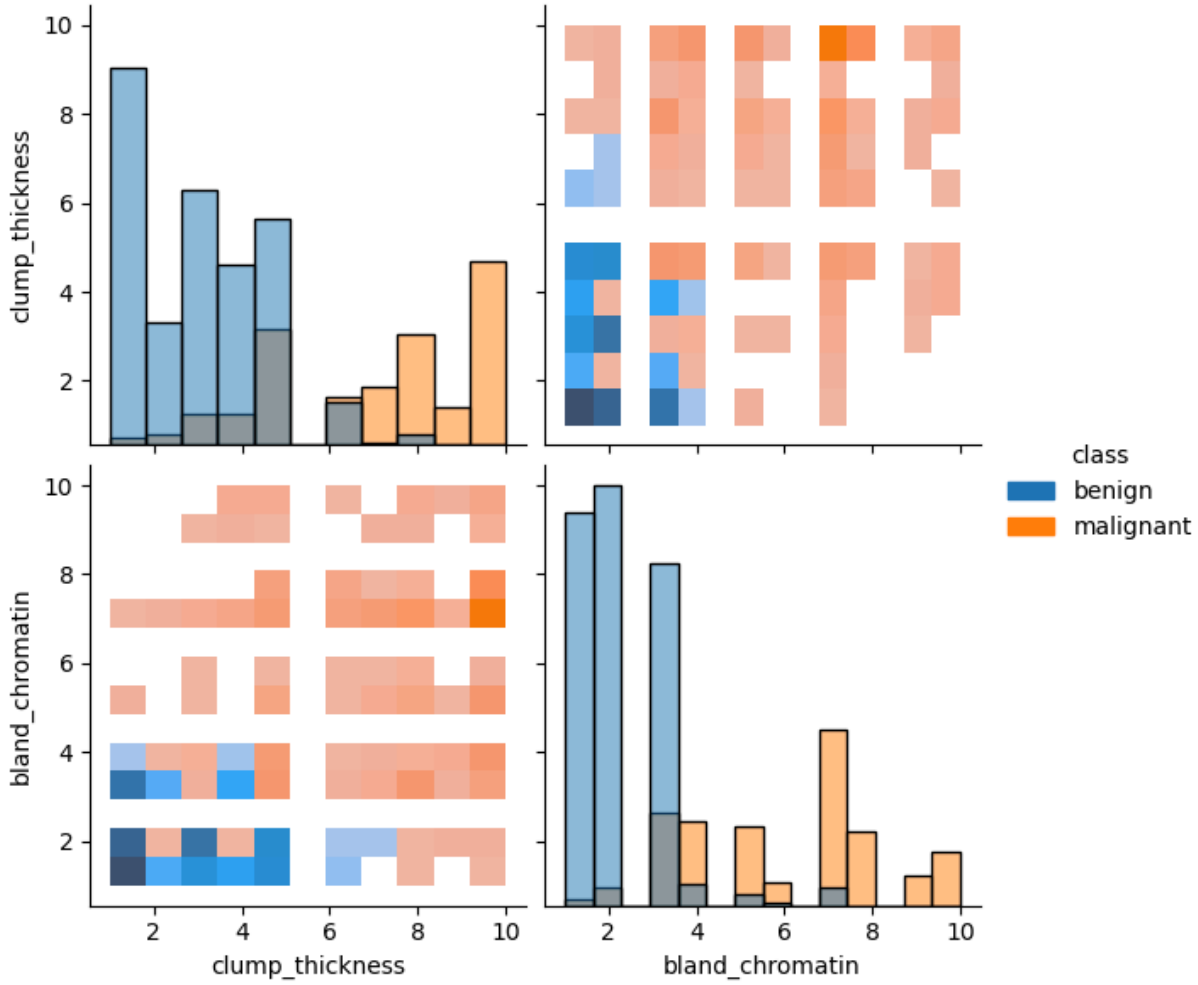


Make a pairplot of our two measurement variables grouped by tumor type and with `kind – 'hist'` in order to make histograms in the off-diagonal plots.

In [56]:
```
sns.pairplot(data=bcd2, vars=["clump_thickness", "bland_chromatin"], hue="cl
    height=3, corner=False)
```

```
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: Future
Warning: use_inf_as_na option is deprecated and will be removed in a future
version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: Future
Warning: use_inf_as_na option is deprecated and will be removed in a future
version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: Future
Warning: use_inf_as_na option is deprecated and will be removed in a future
version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: Future
Warning: use_inf_as_na option is deprecated and will be removed in a future
version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: Future
Warning: use_inf_as_na option is deprecated and will be removed in a future
version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: Future
Warning: use_inf_as_na option is deprecated and will be removed in a future
version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```

Out[56]: <seaborn.axisgrid.PairGrid at 0x175ec5750>



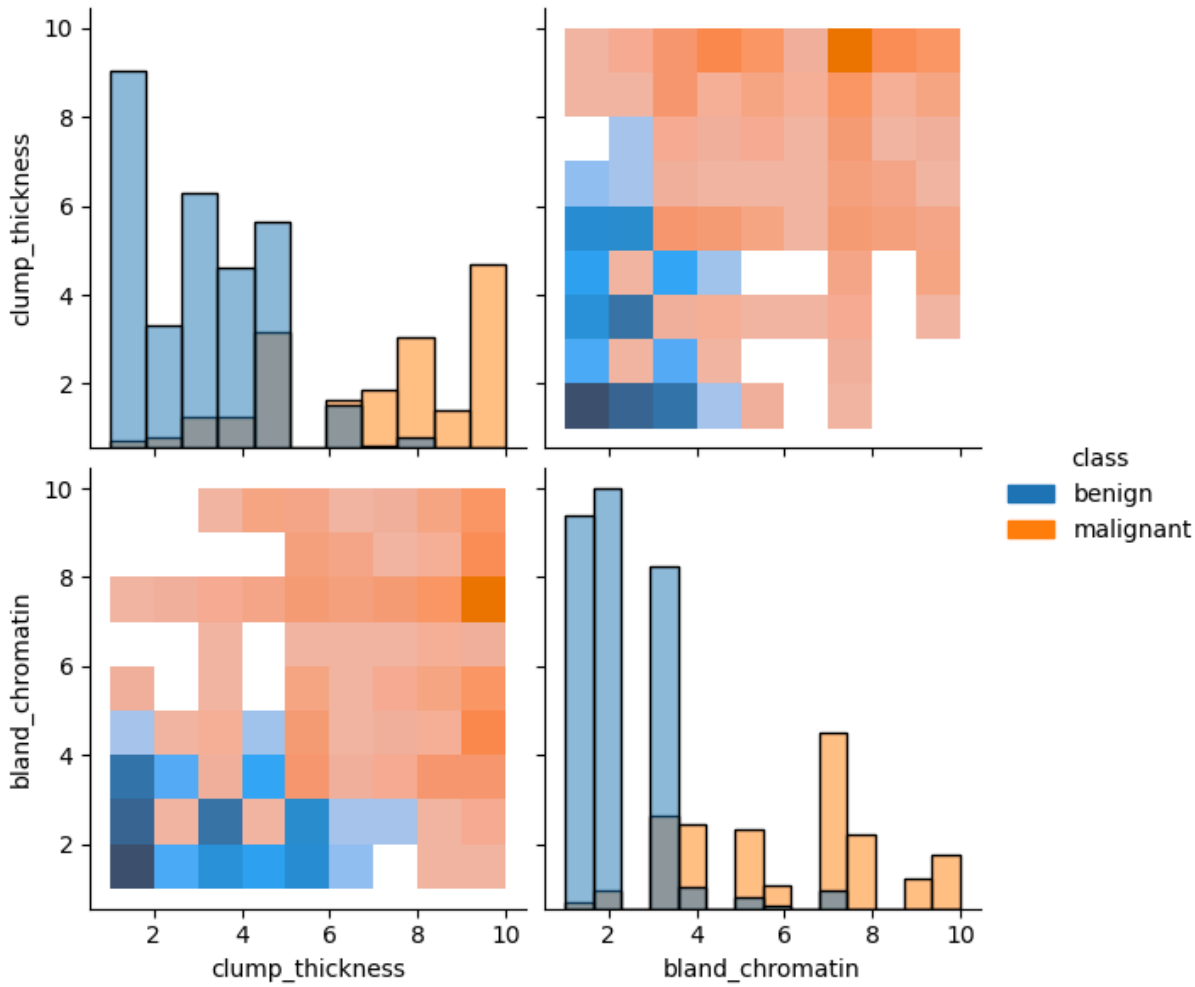
The plot above is okay except for the white space in the diagonal plots that aren't actually missing data. To fix this, we need to set our binwidth to 1, so that it matches the data (which are integers from 1 to 10).

Use the cell below to remake the plot with a binwidth of 1 for the diagonal plot. Hint: use the `plot_kws` argument to adjust this.

In [57]:
```
sns.pairplot(data=bcd2, vars=["clump_thickness", "bland_chromatin"], hue="cl
    plot_kws={"binwidth": 1}, height=3, corner=False)
```

```
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: Future
Warning: use_inf_as_na option is deprecated and will be removed in a future
version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: Future
Warning: use_inf_as_na option is deprecated and will be removed in a future
version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: Future
Warning: use_inf_as_na option is deprecated and will be removed in a future
version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: Future
Warning: use_inf_as_na option is deprecated and will be removed in a future
version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: Future
Warning: use_inf_as_na option is deprecated and will be removed in a future
version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
/opt/anaconda3/lib/python3.11/site-packages/seaborn/_oldcore.py:1119: Future
Warning: use_inf_as_na option is deprecated and will be removed in a future
version. Convert inf values to NaN before operating instead.
  with pd.option_context('mode.use_inf_as_na', True):
```

Out[57]:
```
<seaborn.axisgrid.PairGrid at 0x1761602d0>
```



Your conclusions

In the cell below, briefly state your conclusions from our analysis above. Are either or both of the measurements related to tumor type?

Both variables seems to be indicators of tumor type

Summary

In this tutorial, we learned to analyze data by group:

- the split-apply-combine concept
- grouping using `groupby()`
- doing simple grouped calculations
- doing multiple calculations with `agg()`
- multiIndexing

- simple summaries with pivot tables